

บทที่ 3

วิธีการออกแบบโปรแกรมจำลองการทำงานวงจรดิจิทัล

ข้อดีของโปรแกรมจำลองการทำงานแบบที่มีอยู่นั้น คือ ใช้งานได้ง่าย ด้วยการที่ใช้งานได้ง่ายนี้ จึงทำให้การพัฒนาฮาร์ดแวร์เป็นไปได้โดยไม่ราบรื่นนัก และ เป็นเหตุให้การออกแบบ และ พัฒนาฮาร์ดแวร์นั้นเป็นไปได้อย่างช้า ๆ ไม่รวดเร็วเท่าที่ควร

การแก้ปัญหาข้างต้นนี้ เราจึงออกแบบ และ พัฒนาโปรแกรมจำลองการทำงานขึ้นมาใหม่ ให้มีความสะดวก และ คล่องตัวในการใช้งานที่มากขึ้น พร้อมทั้งทำให้ ผู้ใช้ สามารถใช้งานได้ อย่างง่ายดาย ไม่ว่าจะเป็น User ที่มีความชำนาญในการออกแบบอยู่แล้ว หรือ ผู้ใช้ที่เป็นผู้ใช้ระดับ เริ่มต้น ซึ่งไม่เคยออกแบบ หรือ พัฒนาฮาร์ดแวร์ใด ๆ มาเลย ก็สามารถใช้งานได้อย่างราบรื่น

ในบทที่ 3 นี้จะกล่าวถึงการออกแบบโปรแกรมจำลองการทำงานของวงจรดิจิทัล และ แนวคิดต่าง ๆ ที่นำมาใช้ในการออกแบบ พร้อมทั้งแนวคิดต่าง ๆ ในการจำลองการทำงานวงจรดิจิทัล

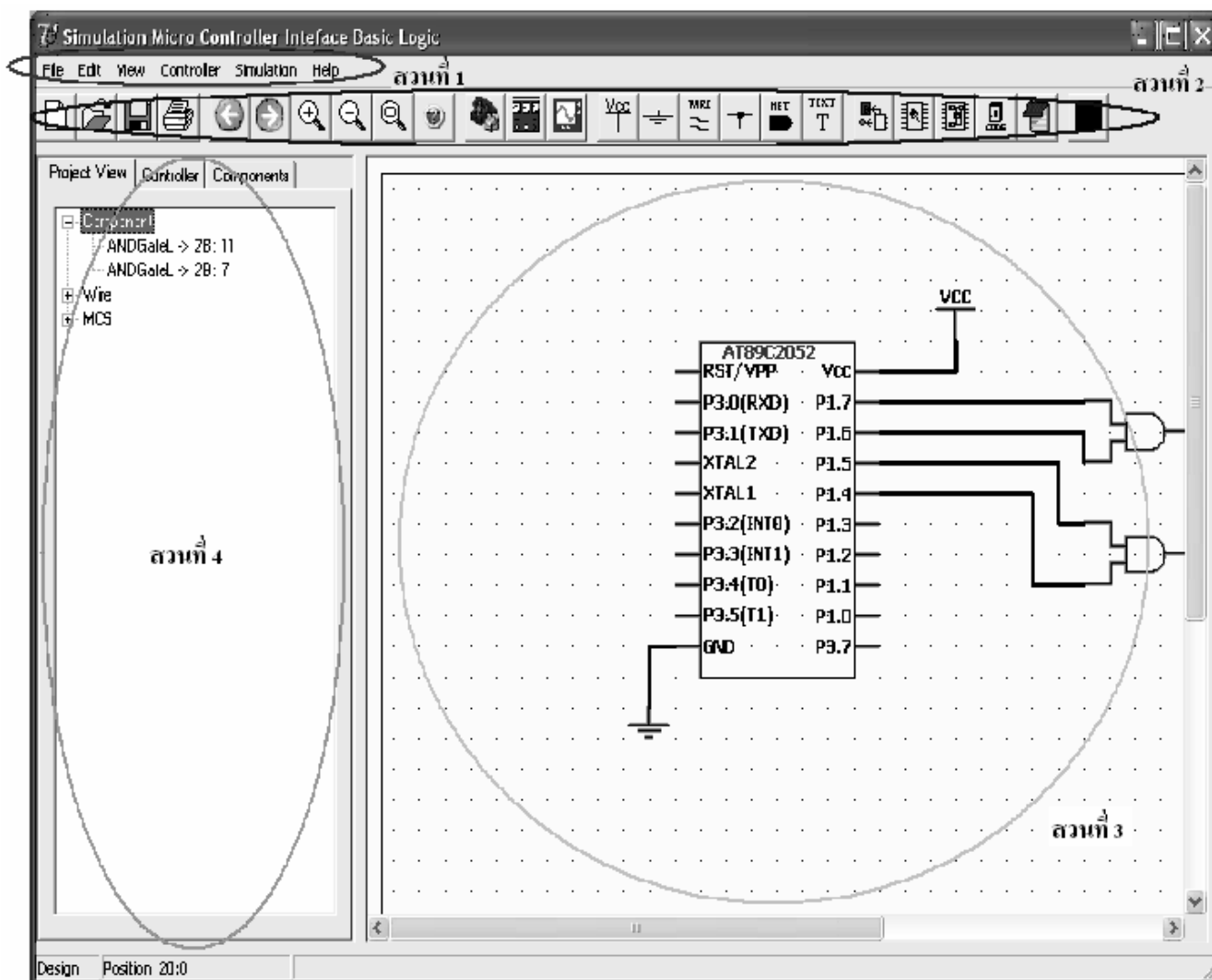
3.1 การออกแบบโครงสร้างการทำงานทางโปรแกรม

3.1.1 การออกแบบส่วนติดต่อผู้ใช้ (User interface)

ในการออกแบบส่วนติดต่อผู้ใช้นั้น ได้แบ่ง ออกเป็นส่วน ๆ ได้ 4 ส่วนดังนี้ คือ

1. ส่วนของ Menu Bar (ส่วนที่ 1)
2. ส่วนของ Tool Bar (ส่วนที่ 2)
3. ส่วนของ Circuit Board (ส่วนที่ 3)
4. ส่วนของ Component Bar (ส่วนที่ 4)

ซึ่งลักษณะหน้าต่างของโปรแกรมจะมีลักษณะดังรูปที่ 3.1



รูปที่ 3.1 แสดงลักษณะหน้าต่างของโปรแกรมจำลองการทำงานที่ได้ออกแบบไว้

3.1.1.1 ส่วนที่ 1 (เมนูบาร์) จะมีเมนูให้เลือกดังนี้

- File
 - ☐ New ใช้ในการสร้าง Project ใหม่
 - ☐ Open ใช้ในการเปิด Project เดิมมาพัฒนาต่อ
 - ☐ Save ใช้ในการเก็บ Project ไว้พัฒนาต่อ
 - ☐ Save as ใช้ในการเก็บ Project ไว้พัฒนาต่อโดยไม่ Save ทับต้นฉบับเดิม
 - ☐ Print ใช้ในการพิมพ์วงจรที่ได้พัฒนาไว้
 - ☐ Exit ออกจากโปรแกรม

- Edit
 - ☐ Routing ใช้ในการเรียกหน้าต่างแสดงผลการเชื่อมต่อ
 - ☐ Error Report ใช้ในการเรียกหน้าต่างแสดงความผิดพลาดของวงจร
- View
 - ☐ Zoom In ใช้ในการขยายวงจรที่กำลังพัฒนา
 - ☐ Zoom Out ใช้ในการย่อขนาดวงจรที่กำลังพัฒนา
 - ☐ Fit Screen ใช้ในการปรับเป็นขนาดที่พอดีหน้าจอ
 - ☐ Simulation ใช้ในการเปิดหน้าจอจำลองการทำงานขึ้นมา
 - ☐ Memory ใช้ในการเปิดหน้าต่างแสดงข้อมูลในหน่วยความจำของ MCS
 - ☐ Register ใช้ในการเปิดหน้าต่างแสดงข้อมูลใน Register ออกมาดู
 - ☐ Hex Code ใช้ในการเปิดหน้าต่างสำหรับจัดการกับการโหลดไฟล์ .Hex
 - ☐ Report ใช้ในการเปิดหน้าต่างสำหรับแสดงรายงานการทำงานของ MCS
 - ☐ Show All ใช้เปิดหน้าต่างทั้งหมดที่กล่าวมา
- Controller
 - ☐ Properties ใช้ในการกำหนดค่าการทำงานและคุณสมบัติของ Controller
- Simulation
 - ☐ Run ใช้ในการจำลองการทำงานแบบต่อเนื่อง
 - ☐ Step Run ใช้ในการจำลองการทำงานแบบเป็น Step
 - ☐ Pause ใช้ในการหยุดการจำลองการทำงานแบบชั่วคราว
 - ☐ Stop ใช้ในการยกเลิกการจำลองการทำงาน
 - ☐ Reset ใช้ในการเริ่มการจำลองการทำงานใหม่ อีกครั้ง
- Help
 - ☐ Simulate Help ใช้ในการดูข้อมูลอธิบายการใช้ Simulate
 - ☐ Controller Help ใช้ในการดูข้อมูลอธิบายการใช้ Controller

3.1.1.2 ส่วนที่สอง (Tool Bar)

	ใช้ในการสร้าง Project ใหม่
	ใช้ในการเปิด Project เดิมมาพัฒนาต่อ
	ใช้ในการเก็บ Project ไว้พัฒนาต่อ
	ใช้ในการพิมพ์วงจรที่ได้พัฒนาไว้
	ใช้ในการขยายวงจรที่กำลังพัฒนา
	ใช้ในการย่อขนาดวงจรที่กำลังพัฒนา
	ใช้ในการปรับเป็นขนาดที่พอดีหน้าจอ
	ใช้ในการดูข้อมูลอธิบายการใช้งานโปรแกรม
	ใช้เรียกใช้งาน Function Generator
	ใช้เรียกใช้งาน Scope
	ใช้เลือกใช้งาน แหล่งจ่ายไฟ
	ใช้เลือกใช้งาน กราวด์
	ใช้ในการเลือกอุปกรณ์ในการเชื่อมต่อสาย
	เลือกใช้ Junction ในการเชื่อมต่อสายเข้าด้วยกัน
	เลือกใช้ Net เพื่อเป็นทางเชื่อมต่อแทนสาย
	ใช้ในการสร้างลาเบลเพื่อคอมเม้นอุปกรณ์
	ใช้ในการเปิดหน้าจอจำลองการทำงานขึ้นมา
	ใช้ในการเปิดหน้าต่างแสดงข้อมูลในหน่วยความจำของ MCS
	ใช้ในการเปิดหน้าต่างแสดงข้อมูลใน Register ออกมาดู
	ใช้ในการเปิดหน้าต่างสำหรับจัดการกับการโหลดไฟล์ .Hex
	ใช้ในการเปิดหน้าต่างสำหรับแสดงรายงานการทำงานของ MCS
	ใช้ในการเลือกสีของสายที่ใช้เชื่อมต่อวงจร

3.1.1.3 ส่วนที่ 3 Circuit Board

จะเป็นส่วนที่ใช้ในการสร้างวงจร เพื่อที่จะนำมาทำการ Simulate การทำงานของวงจร จะเป็นหน้าว่างเปล่า ซึ่งมีจุด Grid เพื่อช่วยบอกพิกัดในการวางอุปกรณ์ และ เชื่อมต่อสาย ซึ่งอุปกรณ์นั้นจะสามารถหามาได้จากส่วนที่ 4 ทางซ้ายมือ หรือ จาก Tool Bars บางตัว มาใช้งานได้

3.1.1.4 ส่วนที่ 4 (Component Bar)

นั่นเป็นส่วนที่ใช้ในการเลือกอุปกรณ์ลงมาวางเรียงกันในส่วนที่ 3 นั้นเอง ซึ่งจะแบ่งเป็น 3 ช่อง ย่อย ๆ ได้แก่

O Project View

ในส่วนนี้จะเป็นส่วนที่บอกรายละเอียดในการเชื่อมต่อ และ บอกรายละเอียดของอุปกรณ์ที่เราเนามาวางว่ามีอะไรบ้าง และ อยู่ที่พิกัดไหน

O Controller

ในส่วนนี้จะเป็นส่วนที่จะใช้ในการเลือกอุปกรณ์ที่เป็นคอนโทรลเลอร์ ซึ่งทำไว้เพื่อพัฒนาต่อ ในภายหลังหน้า

O Components

ในส่วนนี้จะเป็นส่วนที่จะใช้ในการเลือกอุปกรณ์ที่เป็นอุปกรณ์ที่นำมาต่อรวม ซึ่งจะแบ่งออกเป็น 5 ประเภทย่อย คือ

- Basic Gate ได้แก่อุปกรณ์จำพวก Gate ทั่วไป
- Flip Flop ได้แก่อุปกรณ์จำพวก ที่มีการเก็บสถานะได้
- TTL ได้แก่อุปกรณ์จำพวกที่เป็นรูปแบบ IC
- I/P Device ได้แก่อุปกรณ์จำพวกที่เป็นอุปกรณ์ที่รับข้อมูลจาก User
- O/P Device ได้แก่อุปกรณ์จำพวกที่แสดงผลการทำงานให้ User ทราบ

3.1.2 ลำดับการทำงานโดยรวมของระบบ

- สร้างและกำหนด Library กับอุปกรณ์เชื่อมต่ออื่นๆ
- สร้างวงจรและทำการเชื่อมต่อความสัมพันธ์
- สร้างเส้นทางการเชื่อมต่อ
- สร้างสมการการทำงานจากการเชื่อมต่อ
- จำลองการทำงานและแสดงผล

3.2 หลักการออกแบบ library

การออกแบบ library นั้นมีแนวคิด และ ขอบเขตว่าอุปกรณ์ ชนิดใด ๆ ที่ต้องการสร้าง ใช้งาน ต้องประกอบไปด้วย ข้อมูลเบื้องต้น ตำแหน่งการเชื่อมต่อ และ รูปแบบการทำงานโดย ส่วนประกอบแต่ละส่วนนั้นจะมีโครงสร้าง และ หลักการดังต่อไปนี้

3.2.1 ข้อมูลเบื้องต้นของอุปกรณ์ (Library Component)

- Name ชื่ออุปกรณ์ โดยมีความสัมพันธ์เป็น ตัวบ่งชี้ประจำอุปกรณ์ ซึ่งใช้ในการอ้างอิงถึง
- Type ชนิดของอุปกรณ์ เพื่อจำแนกประเภท หรือ จัดกลุ่ม
- Pin จำนวนขาที่สามารถใช้ได้ในการแสดงผล เพื่อออกแบบ
- BmpName ไฟล์ภาพที่ใช้ในการแสดงผล เพื่อออกแบบ
- Description ข้อมูลเพิ่มเติมเกี่ยวกับอุปกรณ์ชนิดนี้

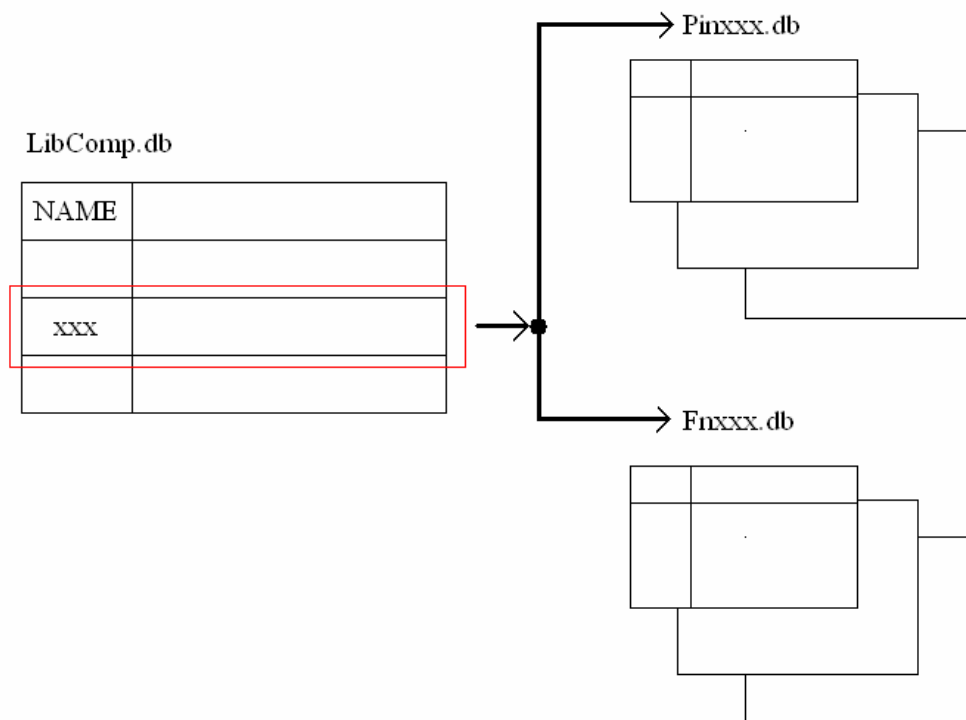
3.2.2 ตำแหน่งสำหรับการเชื่อมต่อ (Pin table)

- PinName ชื่อประจำขาอุปกรณ์
- Type ชนิดการเชื่อมต่อของขาอุปกรณ์ โดยมี
 NC >> ไม่ใช้งาน
 IN >> ขารับสัญญาณ
 OUT >> ขาส่งสัญญาณออก
 Bidirection >> ขาที่สามารถรับ และ ส่งได้พร้อมกัน
- X ตำแหน่งอ้างอิงบนภาพทางแนวนอน
- Y ตำแหน่งอ้างอิงบนภาพทางแนวตั้ง
- Description ข้อมูลเพิ่มเติมประจำขาอุปกรณ์

3.2.3 รูปแบบการทำงาน (Function table)

- PinName [Pins] ชื่ออุปกรณ์ มี Field จำนวนเท่ากับจำนวนขาเชื่อมต่อ
- PresentState ค่าปัจจุบัน ใช้สำหรับตรวจสอบสถานะ
- NextState ค่าในอนาคต ใช้กำหนดสถานะใหม่ให้แก่อุปกรณ์
- EventState การกระทำจากผู้ใช้โดยมี
 MOUSE ON >> เป็นจริงเมื่อ mouse ชี้อยู่บนอุปกรณ์ชนิดนี้
 MOUSE CLICK >> เป็นจริงเมื่อ mouse click บนอุปกรณ์
- Picturename ภาพแสดงผลในสถานะปัจจุบัน

ซึ่งความสัมพันธ์ทางฐานข้อมูลของการจัดเก็บ Libraly จะมีลักษณะดังรูปที่ 3.2

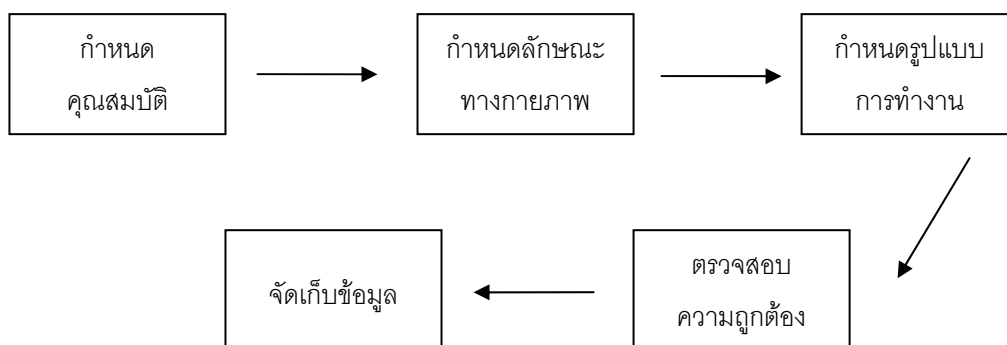


รูปที่ 3.2 แสดงความสัมพันธ์ของข้อมูลในฐานข้อมูลที่ใช้เก็บ Libraly

จากนั้นจึงนำข้อมูลดังกล่าวมาทำตามกระบวนการดังนี้

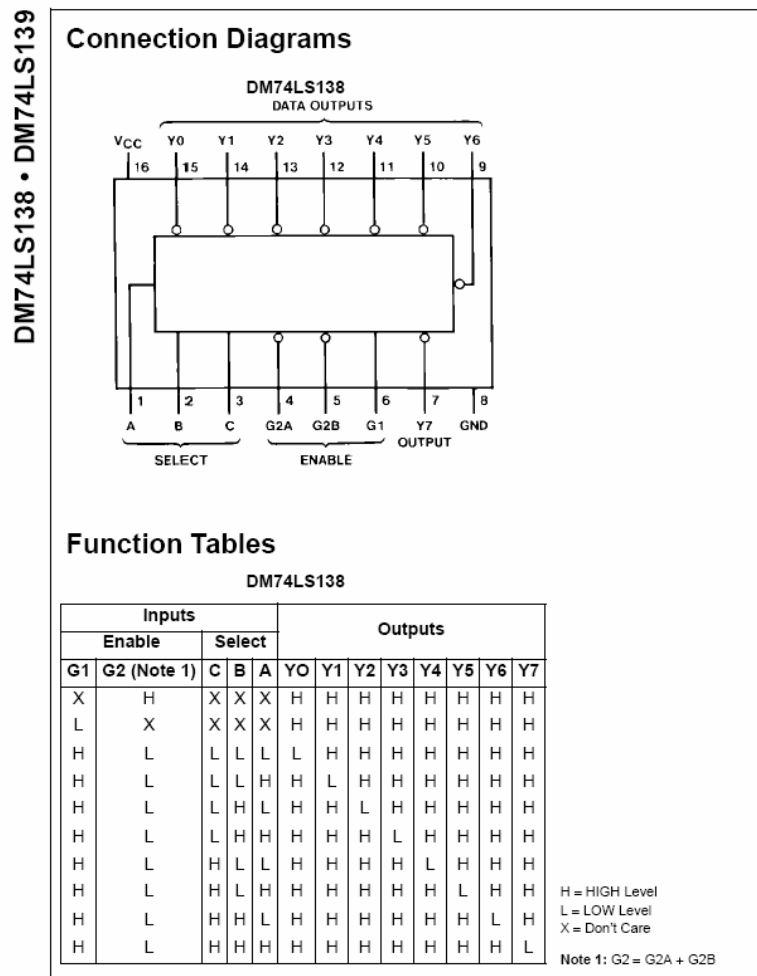
1. กำหนดคุณสมบัติทางกายภาพเบื้องต้น
2. กำหนดคุณสมบัติประจำอุปกรณ์ทุกขา
3. สร้างตารางการทำงาน (truth table) เพื่อกำหนดคุณสมบัติการทำงานของอุปกรณ์
4. ตรวจสอบ การทำงานเพื่อตรวจสอบความถูกต้อง
5. จัดเก็บข้อมูล library

ซึ่งการทำงานทั้งหมดที่กล่าวมานี้จะแสดงออกมาให้เห็นได้ดังรูปที่ 3.3



รูปที่ 3.3 แสดงการทำงานของกระบวนการทำงานเบื้องต้นของโปรแกรมจำลองการทำงาน

ตัวอย่าง การสร้างอุปกรณ์ 74138

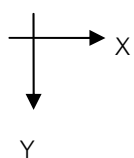


รูปที่ 3.4 แสดงภาพ Datasheet เบอร์ 74138

กระบวนการที่ 1 กำหนดคุณสมบัติทางกายภาพเบื้องต้น

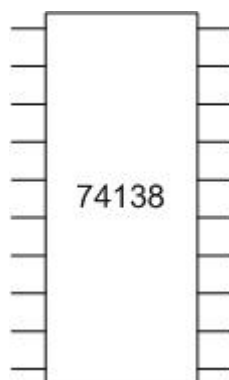
Id	=	00001
Type	=	IC
Name	=	DM74LS138
Picture	=	library/Pic/74138.bmp
Pins	=	16
Description	=	3 – Line – to – 8 – Line Decoder

จากกระบวนการที่ 1 จะได้



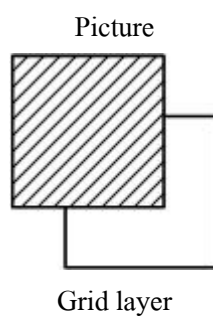
	0	1	2	3	4
0					
1					
2					
3					
4					
5					
6					
7					

Grid layer

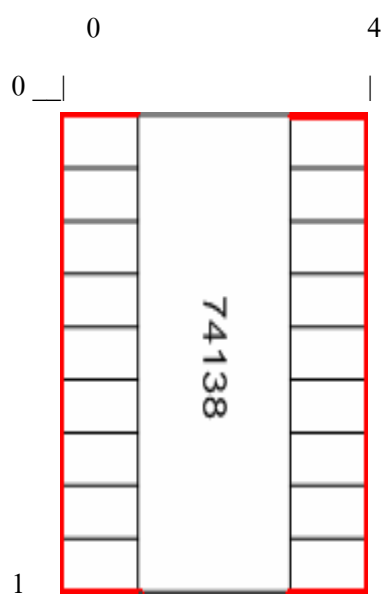
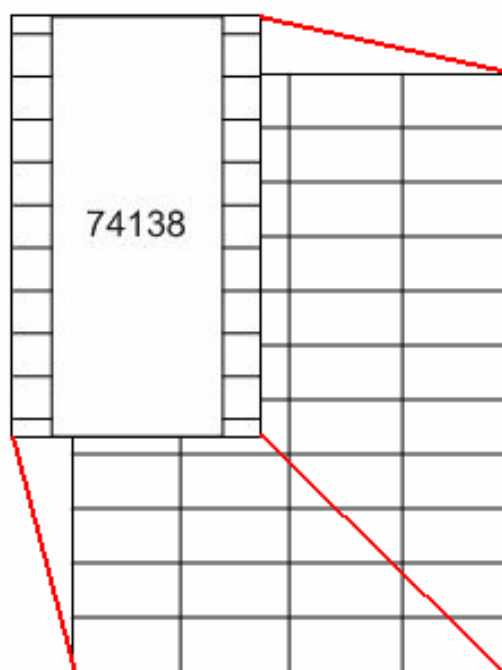


Picture = library / Pic / 74138.bmp

จากนั้นนำภาพมาซ้อน เพื่อให้ได้ซึ่งตำแหน่งขาอุปกรณ์



รูปที่ 3.5 แสดงการหาดำแหน่งพิกัดของขาอุปกรณ์



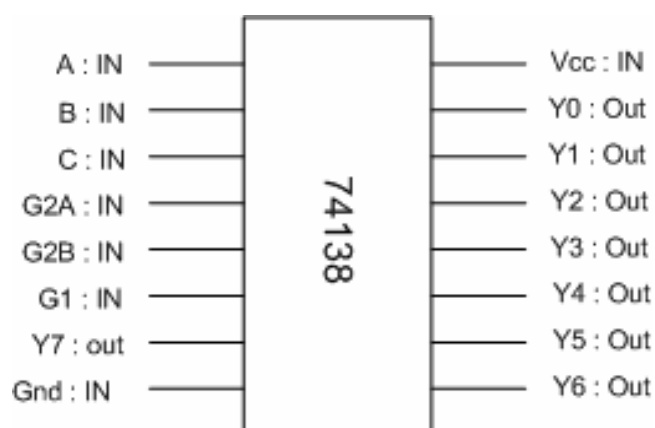
รูปที่ 3.6 แสดงการหาดำแหน่งพิกัดของขาอุปกรณ์(ต่อ)

กระบวนการที่ 2 กำหนดคุณสมบัติของขาอุปกรณ์ทางกายภาพ

Number	Name	Type	Pin	
			X	Y
1	A	IN	0	1
2	B	IN	0	2
3	C	IN	0	3
4	G2A	IN	0	4
5	G2B	IN	0	5
6	G1	IN	0	6
7	Y7	OUT	0	7
8	GND	NC	0	8
9	Y6	OUT	4	1
10	Y5	OUT	4	2
11	Y4	OUT	4	3
12	Y3	OUT	4	4
13	Y2	OUT	4	5
14	Y1	OUT	4	6
15	Y0	OUT	4	7
16	VCC	NC	4	8

รูปที่ 3.7 แสดงตารางคุณสมบัติของขาอุปกรณ์ทางกายภาพ

ถ้าขา VCC กับ GND กำหนด type เป็น NC เพราะโปรแกรมทำงานทางอ้อมคติ



รูปที่ 3.8 แสดงภาพของไอซีที่นำมาเก็บเป็นค่าในตารางในรูปที่ 3.7 ที่ผ่านมา

กระบวนการที่ 3 สร้างรูปแบบการทำงานของอุปกรณ์

Inputs					Output							
Enable		Select										
G1	G2*	C	B	A	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
X	H	X	X	X	H	H	H	H	H	H	H	H
L	X	X	X	X	H	H	H	H	H	H	H	H
H	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	H	H	H	H	H	L	H	H	H	H
H	L	H	L	L	H	H	H	H	L	H	H	H
H	L	H	L	H	H	H	H	H	H	L	H	H
H	L	H	H	L	H	H	H	H	H	H	L	H
H	L	H	H	H	H	H	H	H	H	H	H	L

รูปที่ 3.9 แสดงตารางที่เก็บ Function การทำงานของไอซีที่จะเก็บใน Library

Enable		Select			State Message			
G1	G2*	C	B	A	PresentState	NextState	EventState	Picturename
X	H	X	X	X	0	0	-	-
L	X	X	X	X	0	0	-	-
H	L	L	L	L	0	0	-	-
H	L	L	L	H	0	0	-	-
H	L	L	H	L	0	0	-	-
H	L	L	H	H	0	0	-	-
H	L	H	L	L	0	0	-	-
H	L	H	L	H	0	0	-	-
H	L	H	H	L	0	0	-	-
H	L	H	H	H	0	0	-	-

รูปที่ 3.10 แสดงตารางความสัมพันธ์พิเศษทาง State และ Message ของไอซีที่จะเก็บใน Library

กระบวนการที่ 4

- ตรวจสอบความถูกต้องของ truth table โดย ทำการสร้าง Input ทุกๆเหตุการณ์แล้วทำการ Test เพื่อหา Output หากมีเหตุการณ์ใดไม่สามารถแจกแจงหรือแสดงผลของ Output ได้เพียงเหตุการณ์เดียว แสดงว่าเกิดความผิดพลาด ระบบจะใช้ค่า default ให้แทนพร้อมกับแจ้งข้อผิดพลาด

กระบวนการที่ 5

- เก็บข้อมูลลง library เพื่อจบสมการนี้

3.3 การออกแบบวงจร

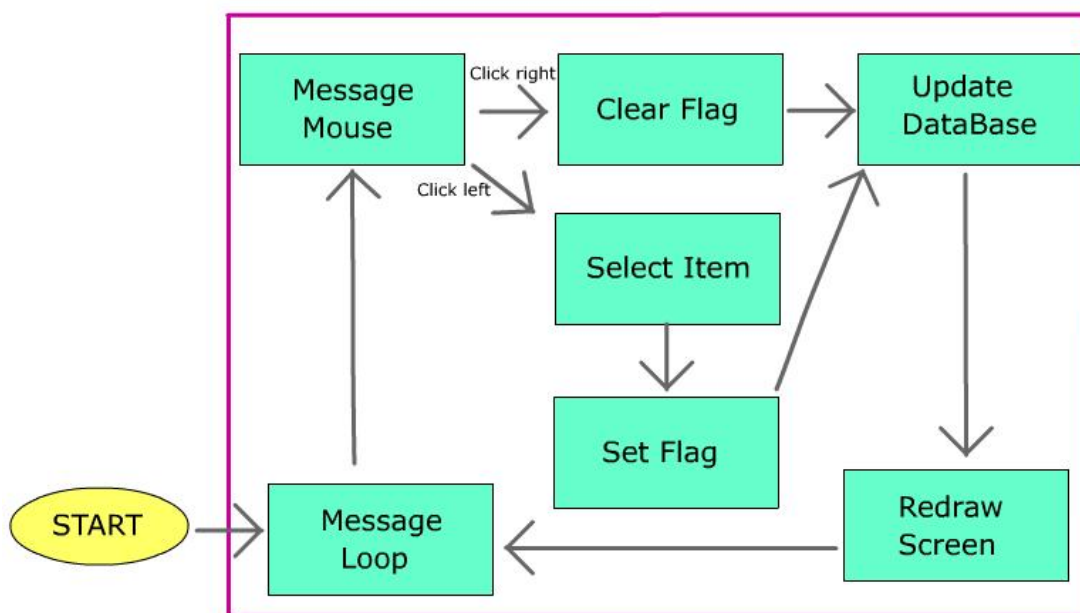
กระบวนการออกแบบวงจรนั้นต้องประกอบไปด้วยข้อมูลหลักดังนี้

- ข้อมูลการจัดวางตำแหน่งและการเชื่อมต่ออุปกรณ์
- หน้าจอเสมือนเพื่อนำอุปกรณ์มาวาดก่อนแสดงผลจริง
- หน้าจอแสดงผลหลัก เพื่อนำภาพจากหน้าจอเสมือนมาแสดงผล

ซึ่งข้อมูลการจัดวางอุปกรณ์ประกอบไปด้วย

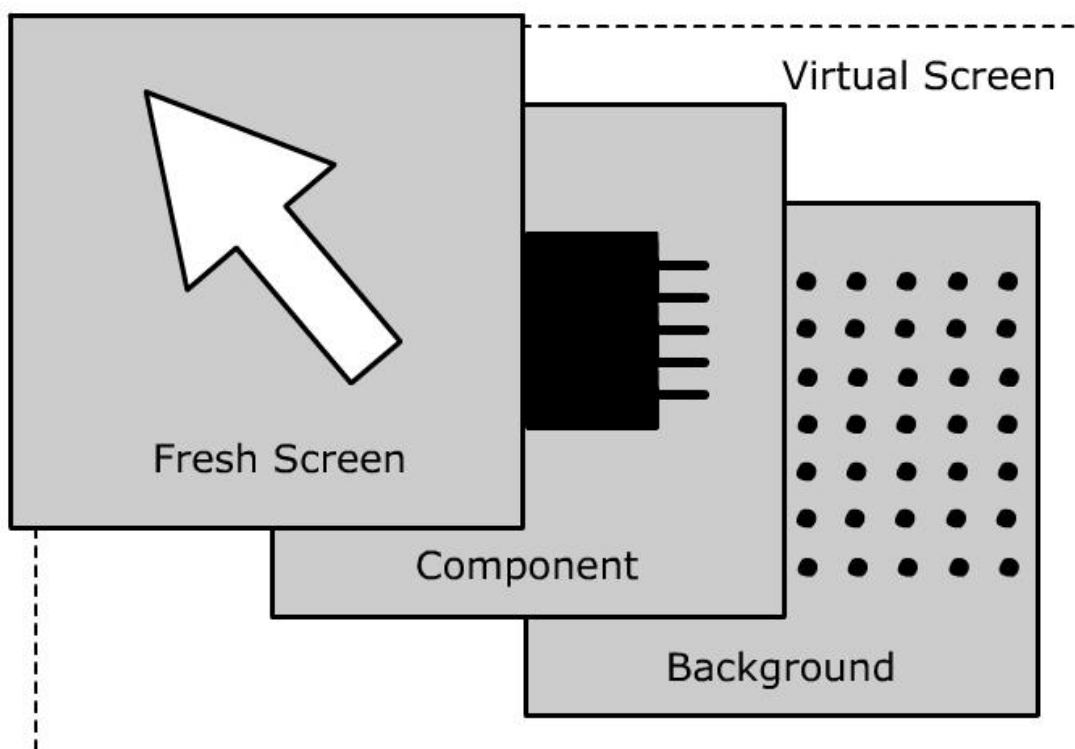
- Serial รหัสประจำตัวอุปกรณ์มีหน้าที่เป็นตัวบ่งชี้อุปกรณ์ในวงจร
- LName ชื่อประจำตัวอุปกรณ์เพื่อระบุอุปกรณ์
- X1 ตำแหน่งอุปกรณ์ทางแนวนอน
- X2 ตำแหน่งอุปกรณ์ทางแนวตั้ง
- X2 ตำแหน่งอุปกรณ์ปลายทางแนวนอน
- Y2 ตำแหน่งอุปกรณ์ปลายทางแนวตั้ง
- Flag ตำแหน่งอุปกรณ์แสดงหมายเลขของอุปกรณ์
 1. อุปกรณ์จาก Library
 2. Vcc
 3. GND
 4. Line จากการลากเส้น
 5. Junction
 6. Net
 7. Text
 8. AT89C52
 9. AT89C2051
- History ลำดับการจัดวางอุปกรณ์

ส่วนในการเลือกอุปกรณ์จะมีแนวคิดในการทำงานของโปรแกรกดังในรูปที่ 3.11



รูปที่ 3.11 แสดงแนวคิดการทำงานของโปรแกรมในส่วนของการลากอุปกรณ์มาวาง

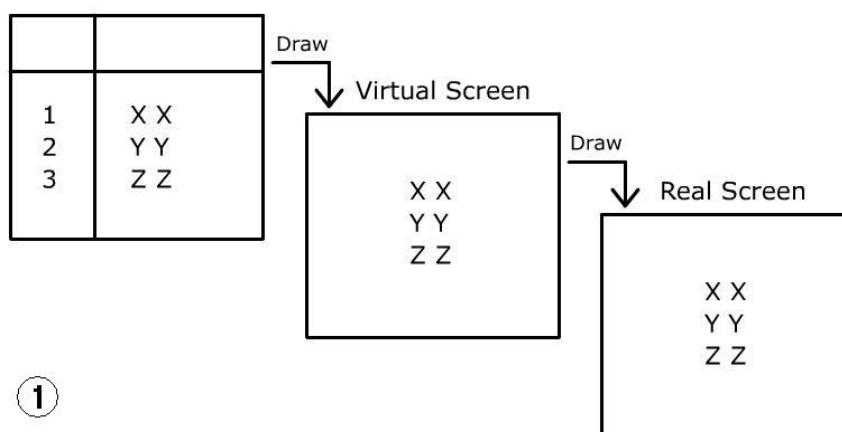
3.3.1 หน้าจอเสมือน



รูปที่ 3.12 แสดงแนวคิดในการออกแบบในส่วนหน้าจอเสมือน

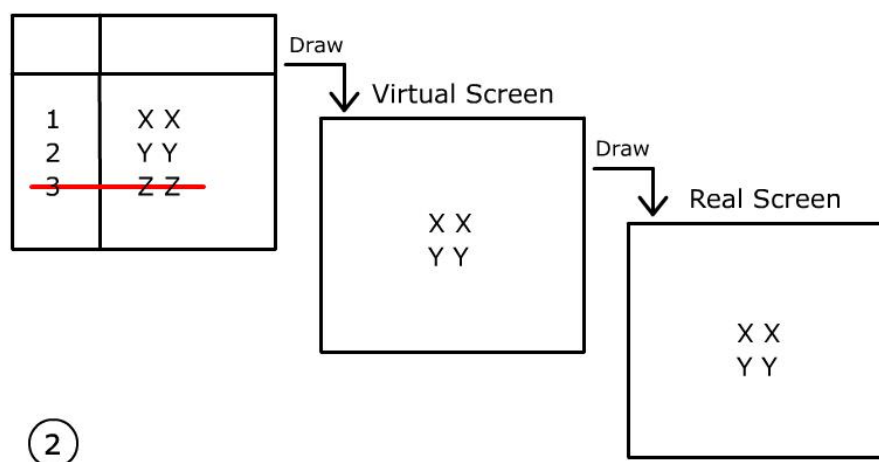
หน้าจอเสมือนทำหน้าที่เป็น หน่วยความจำหน้าจอสำรองก่อนที่จะทำการวาดจริงขึ้นหน้าจอใหม่อีกครั้งโดยจะมีการตรวจสอบ การวาดอยู่ 2 รูปแบบใหญ่ๆ โดยมีการวาดทั้งหน้าจอใหม่จากฐานข้อมูลการจัดวางอุปกรณ์ และการวาดหน้าจอใหม่จากหน่วยความจำล่าสุดซึ่งมีหลักการทำงานดังนี้

- การวาดหน้าจอใหม่จากฐานข้อมูล ใช้ในเพื่อแสดงอุปกรณ์ที่เหลือนหลังจากมีการเปลี่ยนแปลงข้อมูลของอุปกรณ์ในฐานข้อมูล



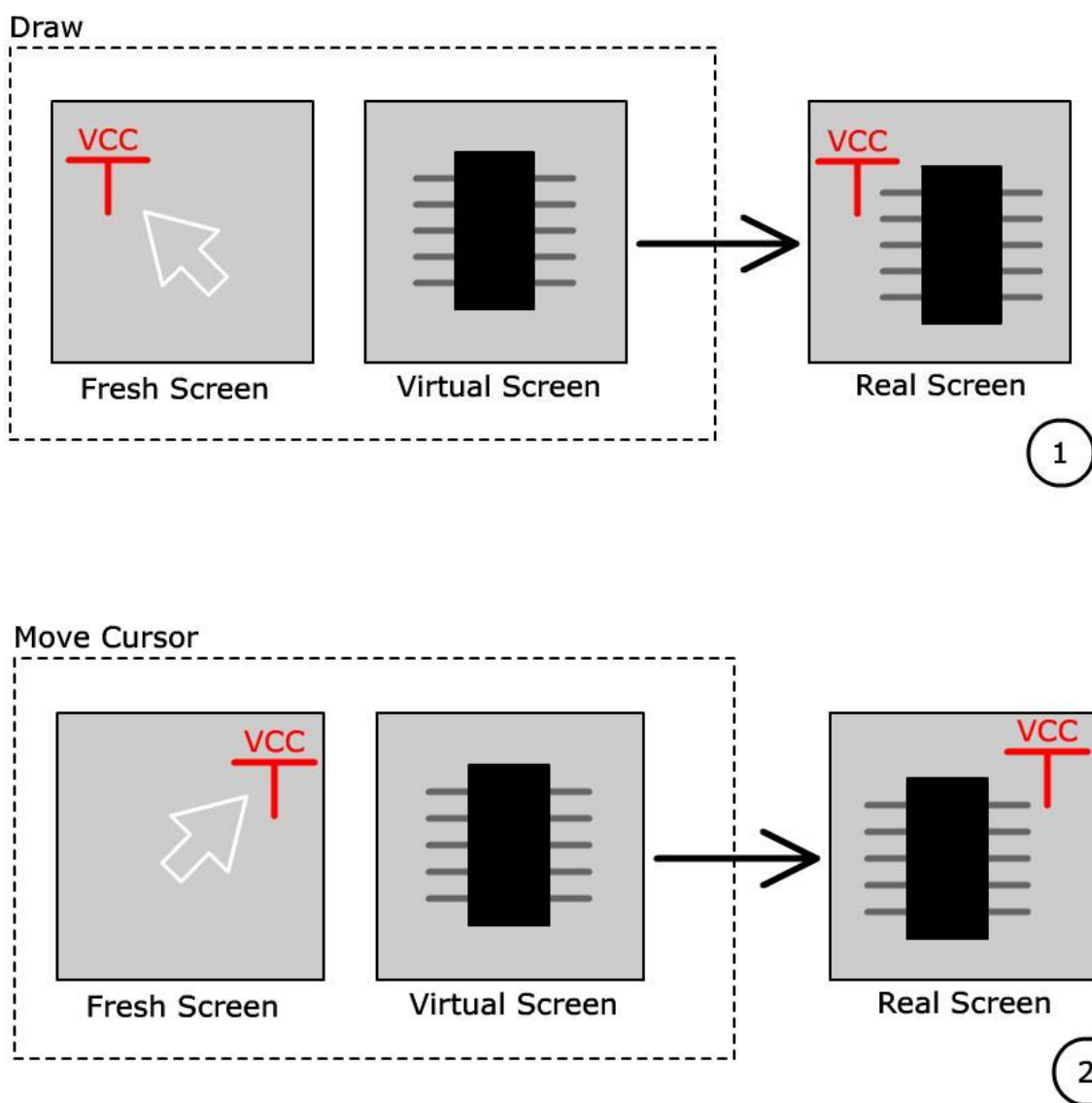
รูปที่ 3.13 แสดงภาพการทำงานของหน้าจอเสมือน

- การวาดหน้าจอใหม่จากหน่วยความจำล่าสุด ใช้เพื่อแสดงผลหลังจากการวาดภาพเดิมๆ ที่ตำแหน่งใหม่ สังเกตได้ว่าที่ Virtual Screen ไม่มีความจำเป็นในการวาดหน้าจอใหม่อีกครั้ง



รูปที่ 3.14 แสดงภาพการทำงานของหน้าจอเสมือน (ต่อ)

เมื่อได้ใช้การออกแบบในส่วนหน้าจอเสมือนมาช่วยแล้ว ก็จะทำให้การทำงานในส่วนของการวาดภาพใหม่เมื่อมีการย้ายที่อุปกรณ์จากที่หนึ่งไปยังอีกที่หนึ่งเป็นไปได้อย่างรวดเร็วยิ่งกว่าเดิมดังภาพที่ 3.15

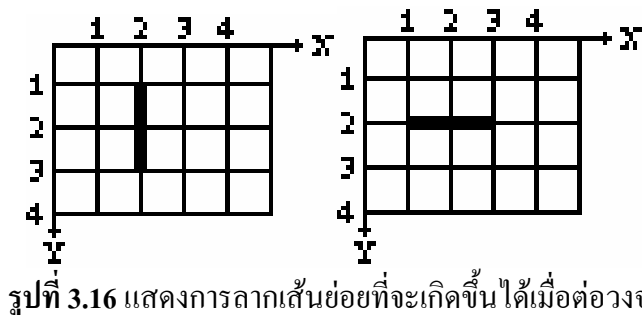


รูปที่ 3.15 แสดงผลการทำงานของหน้าจอเสมือน เมื่อมีการย้ายอุปกรณ์

3.4 การสร้างสมการการเชื่อมต่อ (การค้นหาเส้นทาง)

3.4.1 การสร้างเส้นทาง

สามารถสร้างได้จากการลากสายที่ wire โดยการสร้างเส้นทางมีเงื่อนไข การสร้างเส้นทางว่า จะต้องลากเป็นเส้นตรงเพียงเท่านั้น ซึ่งการเชื่อมต่อแบบเส้นตรงนั้นสามารถตรวจสอบจุดต่อ (Junction) ได้อย่างง่ายและชัดเจนโดยมีโครงร่างการใช้งานดังนี้



รูปที่ 3.16 แสดงการลากเส้นย่อยที่จะเกิดขึ้นได้เมื่อต่อวงจร

Name line (color)

X1 = 2

Y1 = 1

X2 = 2

Y2 = 3

Name Line (color)

X1 = 1

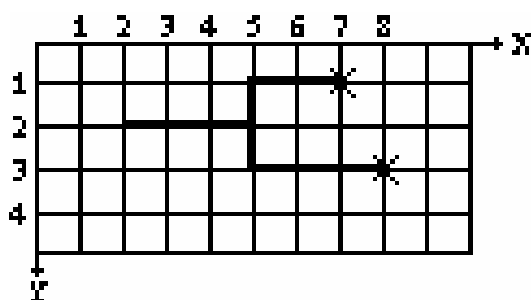
Y1 = 2

X2 = 3

Y2 = 2

รูปที่ 3.17 แสดงข้อมูลการลากเส้นที่ได้จากแต่ละเส้นในรูป 3.16

การค้นหาเส้นทางแบบหลายเส้นทาง



(ก)

Line (2,2 , 5,2)

Line (5,2 , 5,3)

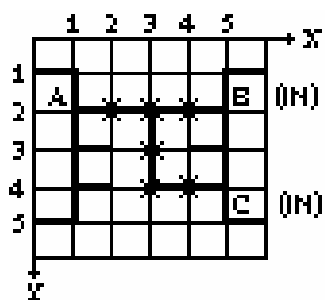
Line (5,3 , 8,3)

Line (5,1 , 7,1)

(ข)

รูปที่ 3.18 (ก)แสดงการลากเส้นในวงจร (ข) แสดงข้อมูลการลากเส้นที่เกิดขึ้น

3.4.2 การค้นหาเส้นทาง

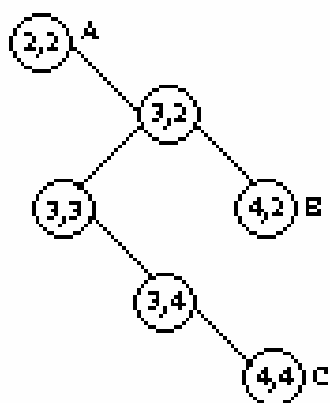


รูปที่ 3.19 แสดงการลากเส้นที่จะเกิดขึ้นได้เมื่อต่อวงจร

จากเส้นทางสามารถนำมาแตก Tree โดยมีเงื่อนไขดังนี้

- การแตก Tree จะไม่ย้อนกลับทางเดิม
- การแตก Tree จะไม่แตก Node ที่เคยมีอยู่แล้ว

Tree ของการค้นหาเส้นทาง



รูปที่ 3.20 แสดง Tree ที่เกิดขึ้นจากการต่อวงจรในรูปที่ 3.19

การแตก Tree จะเห็นได้ว่า

$A \rightarrow B$

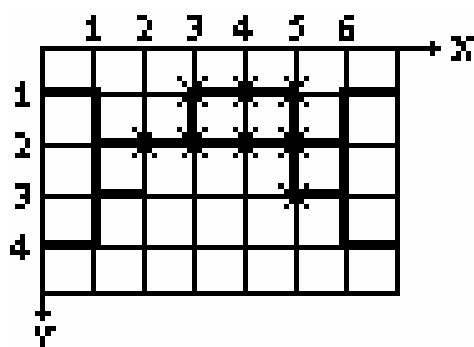
$A \rightarrow C$

นำการแตก Tree มาเก็บเป็นข้อมูลลงฐานข้อมูล

PinName	Relation
B	A
C	A

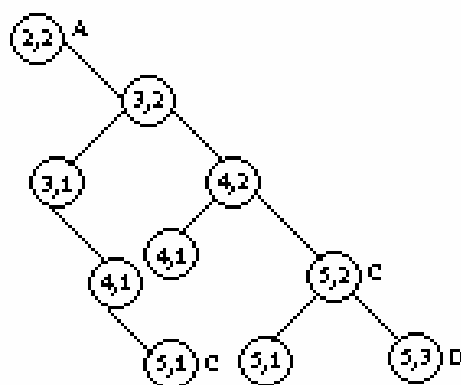
รูปที่ 3.21 แสดงตารางที่จะใช้ในการเก็บข้อมูลการต่อวงจรลงในฐานข้อมูลจริง

3.4.3 การค้นหาเส้นทางแบบมี Loop



รูปที่ 3.22 แสดงการลากเส้นที่จะเกิดขึ้นได้เมื่อต่อวงจรที่มีลักษณะเป็น Loop

ตามเงื่อนไขในการหาเส้นทางในหน้าที่ผ่านมาการแตก Tree จะไม่ทำย้อนกลับไปทางที่เคยทำผ่านมาแล้วก็จะทำให้ Tree ที่ได้ออกมามีลักษณะดังภาพที่ 3.23



รูปที่ 3.23 แสดง Tree ที่เกิดขึ้นจากการต่อวงจรในรูปที่ 3.22

สมการหาเส้นทาง

A->C

A->D

ฐานข้อมูลสมการเส้นทาง

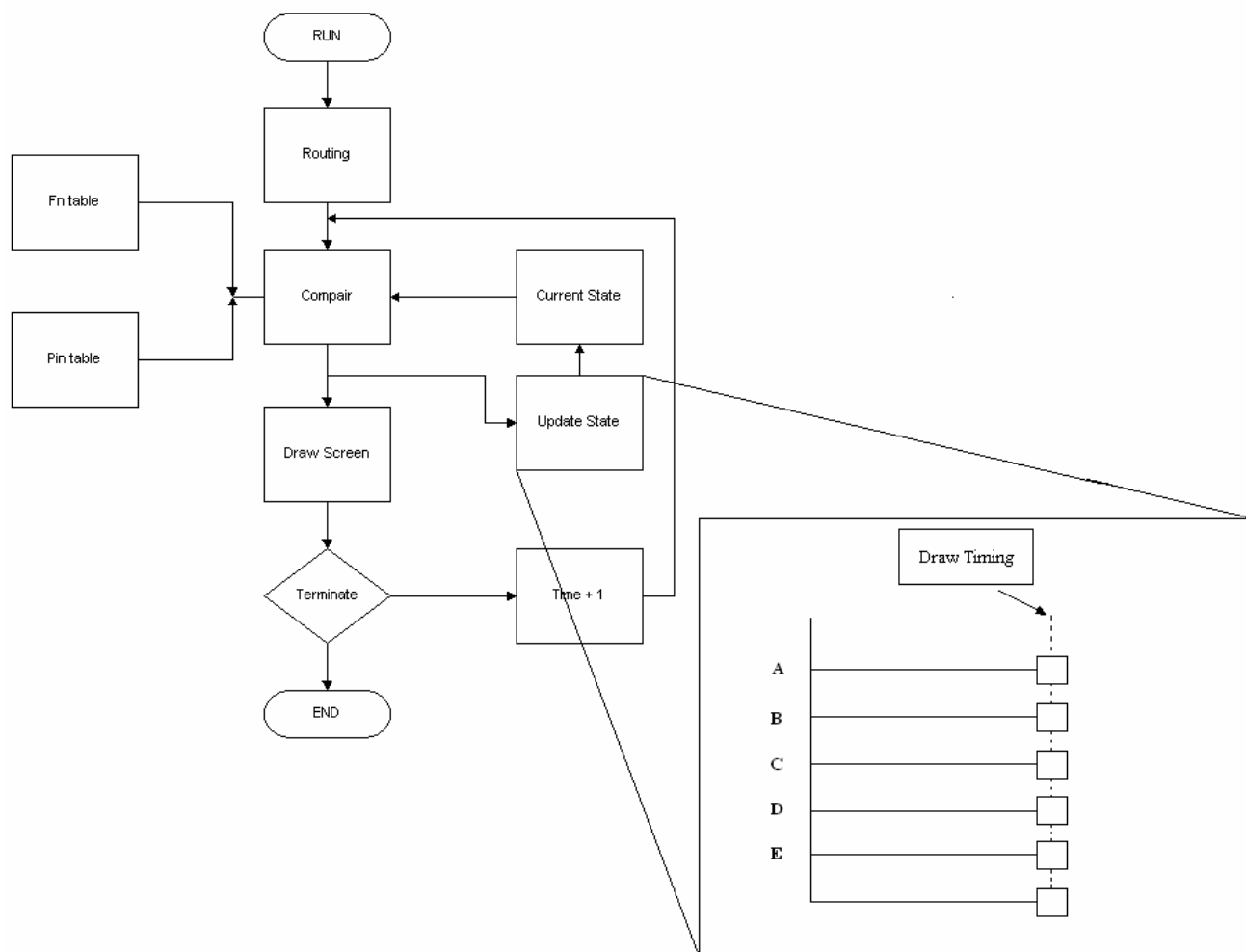
Serial	Pin Name	Relation
2	C	A
2	D	A
1	B	1

สถานะขาลอยมีค่าเป็น 1

รูปที่ 3.24 แสดงข้อมูลที่เกิดขึ้นจากการต่อวงจรในรูปที่ 3.22

3.5 Timing

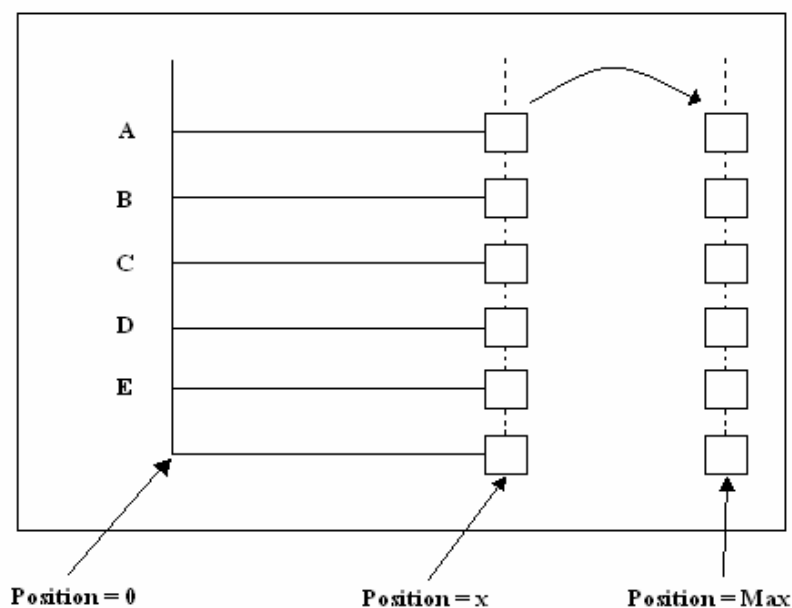
ในส่วนของ Timing นั้นจะมีลักษณะการทำงานดังรูปที่ 3.25



รูปที่ 3.25 แสดงโฟลวชาร์ตการทำงานเมื่อมีการรัน

ซึ่งเมื่อมีการรันจะมีขั้นตอนการทำงานดังนี้ คือ จะทำการค้นหาเส้นทางว่าอะไรต่อกับอะไรมั่ง จากนั้นก็จะนำไปเปรียบเทียบกับตารางความจริงของผลการทำงานในระบบฐานข้อมูล และ Current State เพื่อจะได้ผลออกมาแล้วผลที่ได้นั้นก็จะถูกนำไปอัปเดตState และนำมาวาดลงบนจอไปพร้อม ๆ กัน แล้วเช็คว่ามีการจบโปรแกรมหรือไม่ หากมีการหยุดก็จะหลุดออกไปที่ End ก็เป็นการจบโปรแกรมย่อยส่วนนี้ และ ในขณะเดียวกันที่มีการ Update State เราก็ยังสามารถนำเอาข้อมูลขณะช่วงเวลานั้นมาแสดงผลเป็น Timing Diagram ได้อีกด้วย ซึ่งลักษณะการวาด Timing Diagram จะมีลักษณะดังรูปที่ 3.26

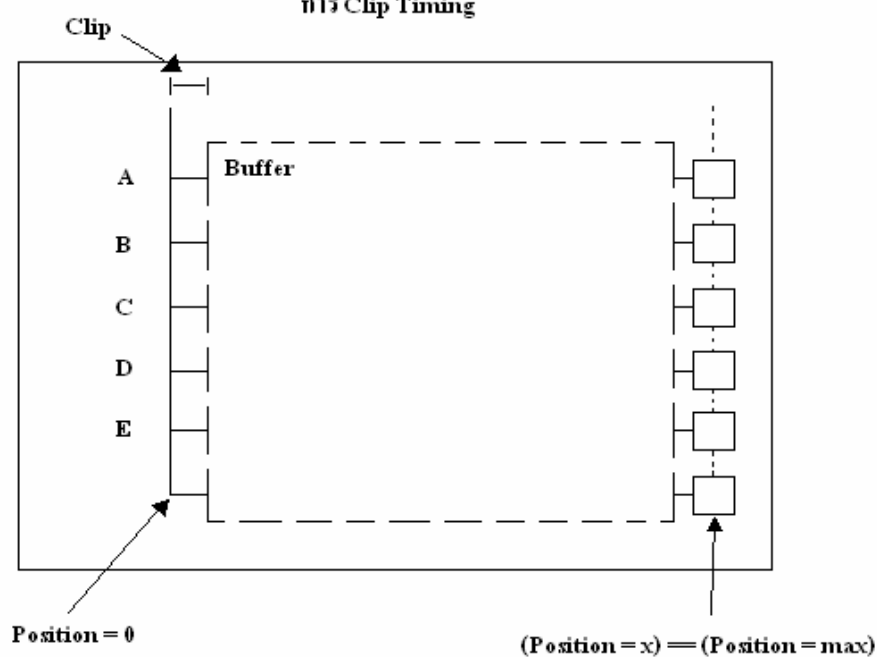
ก15 Draw timing



รูปที่ 3.26 แสดงการวาด timing Diagram

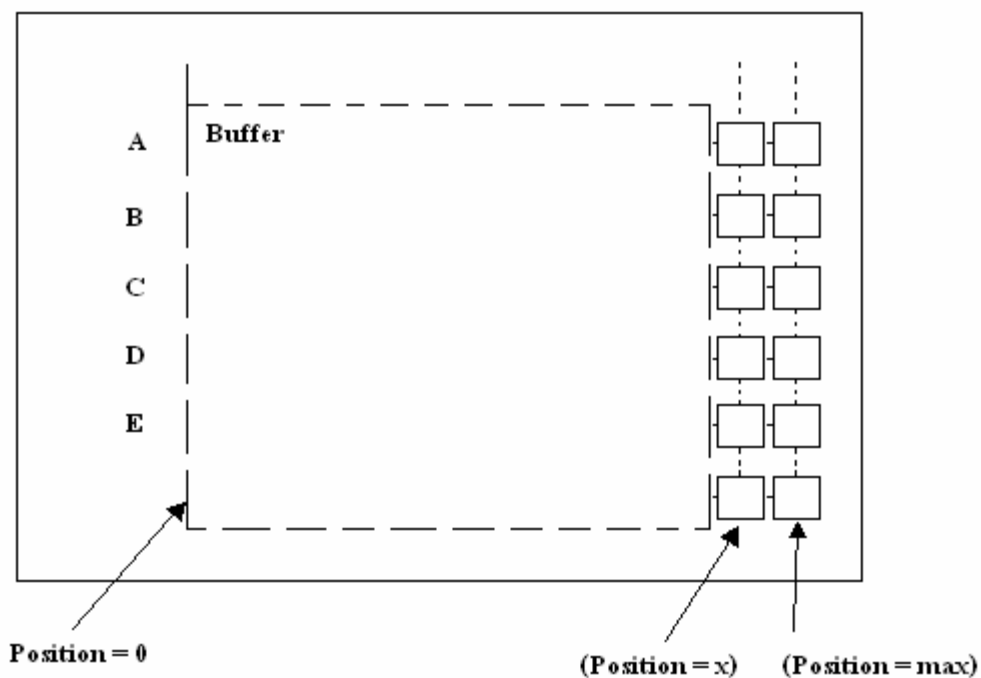
ซึ่งแนวคิดในการวาด Timing Diagram ก็คือ จะวาดข้อมูล ณ เวลาต่าง ๆ โดยข้อมูลเหล่านั้นก็คือ Output ที่ขาอุปกรณ์ต่าง ๆ โดยจะวาดเป็นช่วงเวลาไปเรื่อย ๆ ซึ่งข้อมูล Timing Diagram นี้จะถูกเก็บไว้ในเมมโมรี ที่จำกัดขนาดไว้ ดังนั้นเมมโมรีนี้จึงมีขนาดที่จำกัด ด้วยเหตุนี้เองเมื่อวาดไปจะเมมโมรีเต็มแล้วจึงต้องมีการจัดการเมมโมรีของ Timing Diagram ด้วย ซึ่งจะมีวิธีการดังรูปที่ 3.27

ก15 Clip Timing



รูปที่ 3.27 แสดงการจัดการเมมโมรีของ Timing

ซึ่งขั้นตอนในการจัดการเมโมรีส่วนนี้ก็คือ จะทำการเขียนเมโมรี ณ ช่วงเวลาต่าง ๆ ไปเรื่อย ๆ จนเมื่อ Position x เท่ากับ Position max ก็ทำการก๊อปปี้ส่วนที่ผ่าน ๆ มาตั้งแต่ Position 1 ไปจนถึง Position x แล้วทำการเลื่อนทั้งหมดไปเริ่มใหม่ที่ Position 0 จากนั้นก็จะเหลือพื้นที่สำหรับเขียนเมโมรีเพิ่มขึ้นมาดังรูปที่ 3.28



รูปที่ 3.28 แสดงการจัดการเมโมรีของ Timing เมื่อเมโมรีเต็ม